

LAPORAN TUGAS BESAR

STRATEGI ALGORITMA

IMPLEMENTASI ALGORITMA GREEDY PADA BOT INCESS
DALAM PERMAINAN ROBOCODE TANK ROYALE



Disusun Oleh:

Kelompok Tabolabale

Putu Laura Claudia Ardani (124140021)

Isnaini Febriana (124140075)

Cania Febriyanti (124140153)

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA

2026

DAFTAR ISI

BAB I	
DESKRIPSI TUGAS.....	3
BAB II	
LANDASAN TEORI.....	9
2.1 Dasar Teori.....	9
2.2 Cara Kerja Program.....	9
2.2.1 Cara Implementasi Program.....	10
2.2.2 Menjalankan Bot Program.....	12
2.2.3 Kesimpulan.....	13
BAB III	
APLIKASI STRATEGI GREEDY.....	14
3.1 Proses Mapping Persoalan Robocode Tank Royale menjadi Elemen-elemen Algoritma Greedy.....	14
3.2 Eksplorasi Alternatif Solusi Greedy.....	14
3.3 Analisis Efisiensi dan Efektivitas Solusi Greedy.....	15
3.4 Strategi Greedy yang Dipilih.....	16
BAB IV	
IMPLEMENTASI DAN PENGUJIAN.....	17
4.1 Implementasi Algoritma Greedy pada Program Bot Incess.....	17
4.1.1 Prinsip Desain dan Strategi Inti Incess.....	17
4.1.2 Pseudocode Detail Incess.....	17
4.1.3 Penjelasan Komponen Kunci Implementasi Incess.....	23
4.2 Struktur Data yang digunakan dalam Program Bot Incess.....	24
4.3 Analisis Desain Solusi Algoritma Greedy Melalui Skenario Pengujian.....	27
4.3.1 Skenario Pengujian Bot Asisten.....	27
4.3.2 Ringkasan Kekuatan dan kelemahan Desain Greedy Inces.....	31
BAB V	
KESIMPULAN DAN SARAN.....	33
5.1 Kesimpulan.....	33
5.2 Saran.....	33
LAMPIRAN.....	34
DAFTAR PUSTAKA.....	35

BAB I

DESKRIPSI TUGAS

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari versi asli/pertama permainan ini. Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.

- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa API (Application Programming Interface) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewati turn tersebut. Jika bot melewati turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.

- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:

Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank. Gun digunakan untuk menembakkan peluru dan dapat berputar bersama body atau independen dari body.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama body atau independen dari body.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot. Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok. Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran. Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh. Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan poin sebesar damage yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan poin sebesar 20% dari damage yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan 50 poin.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan 10 poin dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan poin sebesar 2 kalinya damage yang dibuat kepada bot musuh dengan cara menabrak.

- Ram Damage Bonus: Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan poin sebesar 30% dari damage yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

BAB II

LANDASAN TEORI

2.1 Dasar Teori

Algoritma yang digunakan pada pembuatan bot kali ini adalah algoritma greedy. Menurut Roihan et al. (2022), Algoritma greedy merupakan sekumpulan dari algoritma yang selalu mengambil keputusan sementara yang terbaik dalam setiap langkahnya untuk menangani suatu masalah. Prinsip utama dari algoritma ini adalah mengambil sebanyak mungkin apa yang dapat diperoleh sekarang tanpa mempertimbangkan untuk jangka panjang. Menurut Darnita et al. (2019), Algoritma greedy melakukan suatu perhitungan bobot tergantung dari bobot tahapan yang telah dilewati dan bobot itu sendiri

Komponen utama algoritma greedy meliputi:

- Himpunan Kandidat : Semua kemungkinan pilihan yang tersedia pada setiap langkah.
- Fungsi Seleksi : Memilih kandidat terbaik berdasarkan kriteria yang telah ditetapkan.
- Fungsi Kelayakan : Periksa apakah kandidat yang dipilih dapat memberikan solusi.
- Fungsi Objektif : Mengukur nilai berdasarkan solusi yang diperoleh.
- Fungsi Solusi : Menentukan apakah solusi sudah lengkap atau belum.

Dalam permainan Robocode Tank Royale, algoritma greedy digunakan untuk membuat keputusan taktis secara real-time. Menurut Purnama et al. (2024), algoritma greedy akan memilih suatu keputusan yang diambil dahulu untuk solusi alternatif lokal supaya dapat menghasilkan solusi global. Bot harus menentukan kapan harus menyerang, kapan harus menghindari, kapan menembak dengan kekuatan besar, dan kapan menembak ringan semuanya berdasarkan informasi lokal yang tersedia pada setiap turn.

2.2 Cara Kerja Program

Kode program dari bot incess adalah hasil dari kerangka kerja API Robocode Tank Royale, yang dikembangkan menggunakan C# dengan menggunakan target framework versi net10.0. Bot ini bermain secara agresif, bot ini akan mencari lalu mengejar musuh secara terus-menerus untuk menyerang bot musuh secara cepat. Selain itu, bot bergerak dengan pola

zig-zag supaya sulit diprediksi oleh musuh dan bisa menghindari tembakan dari musuh. Strategi ini bisa membuat bot untuk mempertahankan keunggulannya sekaligus meningkatkan peluangnya untuk bertahan hidup dalam pertarungan.

2.2.1 Cara Implementasi Program

a. Inialisasi dan Pengaturan Perilaku Bot

Pada awal program, bot melakukan pengaturan warna body, turret, radar, peluru, dan scan menggunakan library System.Drawing. Selain itu, bot juga mengaktifkan pengaturan:

```
1 AdjustGunForBodyTurn = true;  
2 AdjustRadarForGunTurn = true;
```

Pengaturan tersebut membuat gun tidak ikut berputar saat body bergerak dan radar tetap fokus melakukan scan musuh. Dengan cara ini, akurasi aiming dan pendeteksian target menjadi lebih stabil.

Bot juga memiliki variabel:

```
1 private bool movingForward = true;
```

Variabel ini digunakan untuk menentukan arah gerakan bot ketika melakukan perubahan arah atau saat menabrak dinding.

b. Pergerakan Zigzag

Pada method Run(), bot bergerak menggunakan pola zigzag dengan kombinasi gerakan maju dan belokan kanan-kiri.

Bot melakukan perintah seperti:

```
1 SetForward(180);  
2 SetTurnRight(40);  
3 Go();  
4  
5 SetForward(180);  
6 SetTurnLeft(80);  
7 Go();
```

Pola movement ini digunakan agar arah gerakan bot menjadi lebih sulit diprediksi lawan. Selain itu, movement zigzag membantu bot mengurangi kemungkinan terkena peluru ketika pertandingan berlangsung. Strategi ini termasuk greedy karena bot langsung memilih movement yang dianggap paling aman pada kondisi saat itu tanpa melakukan perencanaan jangka panjang.

c. Penguncian Radar dan Gun

Ketika musuh terdeteksi pada method OnScannedBot(), bot langsung memfokuskan radar, gun, dan body ke arah lawan.

Bot menghitung sudut radar dan gun menggunakan:

```
1 double radarTurn = DirectionTo(e.X, e.Y) - RadarDirection;  
2 double gunTurn = DirectionTo(e.X, e.Y) - GunDirection;
```

Kemudian radar dan gun diarahkan ke target menggunakan:

```
1 SetTurnRadarLeft(NormalizeRelativeAngle(radarTurn));  
2 SetTurnGunLeft(NormalizeRelativeAngle(gunTurn));
```

Pendekatan ini membuat bot dapat mempertahankan target agar tetap berada dalam jangkauan radar dan meningkatkan akurasi tembakan.

d. Pengambilan Keputusan Menembak

Bot menggunakan strategi greedy agresif dalam menentukan kapan harus menembak. Ketika arah gun sudah hampir tepat ke posisi musuh, bot langsung menembak menggunakan fire power maksimum.

Implementasi dilakukan dengan kondisi:

```
1 if (Math.Abs(GunTurnRemaining) < 8)  
2 {  
3     Fire(3);  
4 }
```

Nilai Fire(3) dipilih karena menghasilkan damage terbesar. Bot tidak mempertimbangkan penggunaan energi jangka panjang, melainkan langsung memilih serangan yang paling menguntungkan pada kondisi saat itu.

e. Pengejaran Musuh

Bot juga menerapkan strategi greedy pursuit movement. Jika jarak musuh masih cukup jauh, bot akan langsung bergerak mendekati target.

Implementasi dilakukan menggunakan:

```
1 if (DistanceTo(e.X, e.Y) > 150)  
2 {  
3     SetForward(120);  
4 }
```

Pendekatan ini membantu bot menjaga tekanan terhadap lawan dan meningkatkan peluang peluru mengenai target.

f. Reaksi terhadap Dinding dan Peluru

Ketika bot menabrak dinding, method OnHitWall() akan dijalankan. Bot langsung membalik arah gerakan menggunakan method ReverseDirection() lalu bergerak menjauh dari dinding. Selain itu, ketika terkena peluru pada method OnHitByBullet(), bot akan melakukan dodge movement dengan belokan kanan dan kiri secara cepat. Pendekatan ini membantu bot bertahan lebih lama karena movement menjadi lebih sulit diprediksi lawan.

g. Tidak Ada Perencanaan Jangka Panjang

Bot Incess tidak menggunakan pathfinding, prediksi posisi musuh, ataupun perencanaan strategi jangka panjang.

Semua aksi dilakukan berdasarkan kondisi yang sedang terjadi pada saat itu, seperti:

- langsung mengejar musuh yang terlihat,
- langsung menembak ketika gun sudah tepat,
- langsung menghindar ketika terkena peluru.

Karena itu, pendekatan yang digunakan termasuk algoritma greedy, yaitu memilih aksi yang dianggap paling menguntungkan pada setiap langkah tanpa mempertimbangkan konsekuensi jangka panjang.

2.2.2 Menjalankan Bot Program

Sebelum menjalankan bot, pastikan .NET SDK sudah terinstal karena bot dibuat menggunakan C# dengan menggunakan target framework dengan versi net10.0 , serta aplikasi Robocode Tank Royale dengan versi 10 sudah tersedia sebagai arena permainan. Di dalam folder project harus terdapat lima file utama, yaitu **Incess.cs** yang berisi kode program bot dan **Incess.json** yang menyimpan informasi bot seperti nama, versi, dan deskripsi, karena file JSON tersebut dipanggil langsung oleh program, **project source file** digunakan untuk mengatur konfigurasi project C#, termasuk dependency, versi .NET, dan proses build. **Windows command file** berfungsi untuk menjalankan bot supaya bisa dihubungkan ke terminal. Sedangkan **SH source file** digunakan pada sistem operasi berbasis Linux atau macOS melalui terminal. Berikut ini cara menjalankan bot program :

- **Kompilasi Program:** Bot dikompilasi menggunakan perintah dotnet build pada terminal untuk memastikan apakah kode program yang kita buat terdapat error atau tidak. Apabila build tersebut sudah sukses maka program dapat dijalankan pada .NET.
- **Menjalankan Server:** Jalankan server Robocode Tank Royale sebagai arena tempat bot bertarung. Kita pilih file bot yang akan kita gunakan dahulu lalu klik battle untuk menjalankan bot tersebut.

- Proses Scanning Bot : Bot akan terus melakukan scan menggunakan radar yang berputar sebesar 360 derajat di dalam loop utama melalui perintah `SetTurnRadarRight(360)` untuk mendeteksi musuh. Saat radar berhasil mendeteksi musuh ia akan memanggil method `OnScannedBot()` untuk memberi tahu posisi musuh dalam bentuk koordinat X dan Y. Kemudian bot akan menghitung jarak ke musuh serta sudut yang dibutuhkan untuk mengarahkan radar, gun, dan body ke arah target.
- Evaluasi dan Pemilihan Aksi : Bot akan menghitung jarak ke musuh menggunakan method `DistanceTo()` untuk menentukan apakah perlu mendekati target atau tidak. Lalu bot menghitung sisa sudut putaran gun melalui `GunTurnRemaining` untuk menentukan tembakan itu sudah akurat apa belum. Sehingga jika jarak musuh lebih dari 150 pixel maka bot maju mendekati musuh, dan jika sisa sudut gun kurang dari 8 derajat maka bot langsung menembak dengan power maksimum `Fire(3)`.
- Aksi Bot : Bot bekerja dengan cara terus mendeteksi musuh, bergerak zigzag, lalu menyerang musuh yang terdeteksi. Setelah menentukan aksi, bot langsung bergerak, mengarahkan senjata dan menembak. Bot juga punya reaksi otomatis terhadap kondisi tertentu, misalnya berbalik saat menabrak dinding, menembak kuat saat bertabrakan dengan bot lain, dan bergerak zigzag saat terkena peluru agar sulit ditembak.
- Akhir Permainan: Pertandingan berakhir ketika ronde selesai kemudian program akan menghentikan proses permainan.

2.2.3 Kesimpulan

Algoritma greedy adalah pendekatan yang efektif untuk membuat keputusan taktis secara real-time dalam permainan Robocode Tank Royale. Bot incess mengimplementasikan strategi greedy pada setiap aspek permainan: penembakan, pergerakan, dan mengatur energi kita supaya bisa bertahan lama. Meskipun pendekatan ini tidak selalu menghasilkan solusi optimal, namun sangat efisien dalam mengambil keputusan cepat pada setiap turn dengan informasi yang tersedia.

BAB III

APLIKASI STRATEGI *GREEDY*

3.1 Proses *Mapping* Persoalan Robocode Tank Royale menjadi Elemen-elemen Algoritma Greedy

Dalam implementasi bot incess, strategi greedy diterapkan dengan memetakan permainan Robocode Tank Royale ke dalam elemen-elemen algoritma greedy sebagai berikut:

- **Himpunan kandidat:** Semua aksi yang dapat dilakukan bot itu seperti bergerak, memutar radar, menembak, mengejar musuh, dan menghindari peluru.
- **Himpunan solusi:** Rangkaian aksi yang dilakukan bot selama pertandingan untuk memperoleh skor setinggi mungkin.
- **Fungsi solusi:** Target musuh yang sedang dipilih serta aksi yang dilakukan bot terhadap target tersebut.
- **Fungsi seleksi:** Bot memilih musuh yang berhasil dideteksi radar lalu langsung memfokuskan radar, gun, dan body ke arah lawan.
- **Fungsi kelayakan:** Bot hanya menembak ketika arah gun sudah hampir tepat ke posisi musuh.
- **Fungsi objektif:** Memaksimalkan damage terhadap lawan sekaligus menjaga bot tetap bertahan selama pertandingan berlangsung.

3.2 Eksplorasi Alternatif Solusi *Greedy*

- **Greedy Radar Lock**

Pada strategi greedy ini, bot akan melakukan penguncian pada radar terhadap musuh yang berhasil dideteksi. Setelah target berhasil ditemukan, radar akan terus diarahkan ke posisi lawan agar musuh tidak hilang dari jangkauan scan. Strategi ini membantu bot menjaga fokus terhadap satu target sehingga proses tracking menjadi lebih stabil. Namun, strategi greedy ini juga memiliki kelemahan karena bot menjadi kurang respon terhadap musuh lain yang mungkin berada di posisi yang lebih berbahaya.

- **Greedy High Damage Attack**

Strategi greedy yang kedua ini lebih fokus dengan memberikan damage sebesar mungkin kepada lawan. Bot akan mendekati lawan dahulu sebelum menembak agar peluang mengenai target menjadi lebih besar. Lalu, bot langsung menembak target dengan menggunakan Fire(3) saat arah gun sudah hampir tepat ke target. Strategi ini

dapat menghasilkan serangan yang sangat agresif dan efektif dalam pertarungan jarak dekat. Tetapi strategi ini juga memiliki kelemahan dalam menggunakan fire power maksimum terus-menerus dapat menyebabkan energi bot lebih cepat berkurang apalagi jika banyak tembakan yang meleset.

- **Greedy Pursuit Movement**

Pada strategi yang ketiga ini, bot langsung mendekati musuh ketika jarak lawan masih cukup jauh. Implementasi dilakukan menggunakan kondisi:

```
if (DistanceTo(e.X, e.Y) > 150)
{
    SetForward(120);
}
```

Strategi ini bisa bantu bot dalam menjaga tekanan terhadap lawan dan meningkatkan peluang mengenai target. Namun kelemahan dari strategi ini yaitu bot melakukan movement yang terlalu agresif dapat membuat bot mudah terkena serangan dari musuh, ketika berada terlalu dekat dengan musuh.

- **Greedy Zigzag Dodge**

Strategi ini menggunakan movement zigzag supaya sulit terkena tembakan dari musuh. Bot terus bergerak dengan merubah arah gerak menjadi arah kanan dan kiri, serta melakukan dodge movement tambahan ketika terkena peluru. Pendekatan ini membuat arah gerakan bot lebih sulit diprediksi oleh lawan dan juga membantu bot bertahan lebih lama di arena. Kelemahan dari strategi ini adalah perubahan arah yang sering dilakukan terkadang membuat akurasi tembakan menjadi sedikit kurang stabil.

3.3 Analisis Efisiensi dan Efektivitas Solusi *Greedy*

- **Efisiensi**

Solusi greedy pada bot ini mudah dijalankan karena setiap aksi hanya membutuhkan perhitungan sederhana seperti menghitung arah musuh, sudut radar, dan jarak target. Bot tidak menyimpan riwayat pergerakan lawan dan juga tidak melakukan prediksi yang rumit terhadap musuh sehingga penggunaan memori dan proses komputasinya kecil. Namun, karena strategi greedy yang kita gunakan ini lebih mementingkan kondisi sekarang yang dapat membuat bot kadang menjadi terlalu agresif dan kurang mempertimbangkan kondisi energi atau posisi arena. Contohnya, bot tetap mendekati lawan walaupun darah sedikit atau posisi sedang tidak aman. Selain itu, penggunaan Fire(3) secara terus-menerus memang memberikan damage besar, tetapi energi bot juga cepat habis jika tembakan sering meleset.

- **Efektivitas**

Bot ini menggunakan strategi greedy agresif sehingga keputusan yang diambil itu berdasarkan kondisi terbaik pada saat ini tanpa memikirkan strategi jangka panjang. Saat musuh terdeteksi, bot langsung mengunci radar dan gun ke arah lawan lalu menembak dan juga menabrak musuh menggunakan power besar agar damage yang diberikan maksimal. Dengan menggunakan strategi greedy agresif ini membuat bot efektif dalam membunuh musuh yang memiliki jarak dekat dan juga saat melawan bot musuh yang gerakannya lambat, sehingga bot ini bisa terus menekan musuh dengan serangan agresif dan menjaga target dengan cara tetap dikunci. Bot juga memiliki gerakan zigzag yang membantu mengurangi kemungkinan terkena peluru musuh karena arah gerakannya tidak lurus dan sulit diprediksi. Ketika terkena peluru atau menabrak dinding, bot langsung mengubah arah untuk menghindari serangan berikutnya sehingga mobilitasnya tetap terjaga.

3.4 Strategi Greedy yang Dipilih

Bot Incess menggunakan Strategy Greedy Aggressive Pursuit dan Damage Maximization dengan memperhatikan hal-hal berikut ini:

- **Dasar Pemilihan Strategi Greedy**

Strategi Greedy Aggressive Pursuit dan Damage Maximization dipilih karena penggunaannya yang mudah dan sangat responsif terhadap musuh. Setiap keputusan, seperti lock radar, lock gun, lock body, tembak Fire(3), dekati musuh, lalu tabrak musuh sambil menembak musuh dibuat dengan greedy berdasarkan kondisi sederhana yang mudah diperiksa.

- **Keputusan Penembakan**

Tembak Bot yang selalu memilih Fire(3) dengan kekuatan yang maksimum tanpa mempertimbangkan jarak atau energi. Ini adalah keputusan yang egois dengan mengambil damage terbesar saat ini. Fire(3) menghasilkan 14 poin kerusakan per tembakan dan 9 poin energi balik jika mengenai musuh.

- **Keputusan Gerak Bot**

Bot selalu berada dalam jangkauan efektif dengan mendekati musuh jika jarak lebih dari 150 piksel.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Algoritma *Greedy* pada Program Bot Incess

Pengembangan bot incess dilakukan dengan menerapkan strategi greedy agresif yang berorientasi pada damage maximization dan pursuit movement. Bot Incess menerapkan strategi greedy agresif yang berfokus pada pengambilan keputusan terbaik secara langsung di setiap kondisi pertandingan.

4.1.1 Prinsip Desain dan Strategi Inti Incess

Bot Incess ini dibuat dengan menggunakan konsep permainan yang agresif, yaitu bot selalu mengejar, menabrak, dan menembak musuh selama pertandingan berlangsung. Bot ini tidak terlalu fokus pada pertahanan, tetapi lebih mengutamakan serangan terus-menerus agar lawan kesulitan untuk berkembang. Saat musuh terdeteksi oleh radar, bot langsung mengunci ke arah target, mulai dari radar, gun, hingga body, sehingga fokus serangan tetap terjaga dan musuh tidak mudah lepas dari pantauan. Setelah target terkunci, bot akan menembak menggunakan damage maksimum dan juga menabrak musuh setiap ada kesempatan agar energi lawan cepat berkurang. Selain itu, bot juga terus mendekati musuh untuk menjaga tekanan dalam jarak dekat dan meningkatkan peluang serangan mengenai target. Agar tidak mudah terkena balasan, bot bergerak dengan pola zigzag secara terus-menerus sehingga arah gerakannya lebih sulit diprediksi oleh lawan.

4.1.2 Pseudocode Detail Incess

```
// LIBRARY YANG DIGUNAKAN
// Library dasar C#
using System;
// Library untuk warna bot
using System.Drawing;
// Library utama Robocode Tank Royale
using Robocode.TankRoyale.BotApi;
// Library event seperti scan musuh, kena peluru, dll
using Robocode.TankRoyale.BotApi.Events;
// Bot yang bernama Incess Menggunakan Startegy Greedy
// yaitu Aggressive & Bullet Damage Maksimal
```

```

//
-----
// Bot ini menggunakan strategi greedy agresif.
// Bot selalu memilih aksi terbaik saat ini tanpa mikirin
jangka panjang
// yaitu:
// 1. Mengunci musuh
// 2. Mendekati musuh
// 3. Menabrak Musuh
// 4. Menembak dengan damage maksimum
// 5. Bergerak zigzag agar sulit ditembak
// Heuristik yang digunakan pada Strategy Greedy:
// - Jika musuh terlihat bot akan langsung lock radar &
gun
// - Jika gun hampir tepat bot akan langsung tembak power
besar
// - Jika musuh jauh bot akan mengejar lalu mendekati
musuh
// - Jika kena peluru bot akan dodge zigzag
// Jenis greedy:
// Greedy Aggressive & Bullet Damage Maksimal
public class Incess : Bot
{
    // Variabel yang digunakan untuk mengecek apakah bot
sedang maju atau mundur
    // Yang nantinya akan digunakan pada method
ReverseDirection()
    private bool movingForward = true;
    // =====
    // METHOD MAIN
    // =====
    static void Main()
    {
        // Membuat object bot dan kemudian dijalankan
        new Incess().Start();
    }
    // =====
    // CONSTRUCTOR BOT
    // =====
    // Yang digunakan untuk mengambil informasi bot dari
file JSON
    Incess() : base(BotInfo.FromFile("Incess.json")) { }
    // =====
    // METHOD RUN

```

```

// =====
// Main Method yang nantinya terus berjalan selama bot
hidup
public override void Run()
{
    // PENGATURAN WARNA BOT
    BodyColor = Color.FromArgb(0xFF, 0x69, 0xB4); //
hot pink
    TurretColor = Color.FromArgb(0xFF, 0xFF, 0x00); //
kuning
    RadarColor = Color.FromArgb(0xFF, 0x14, 0x93); //
dark pink
    BulletColor = Color.FromArgb(0x90, 0xEE, 0x90); //
light green
    ScanColor = Color.FromArgb(0xFF, 0xC0, 0xCB); //
light pink
    // =====
    // PENGATURAN GUN & RADAR
    // =====
    // Gun tidak ikut berputar saat body berputar
supaya aim tetap stabil
    AdjustGunForBodyTurn = true;
    // Radar tidak ikut berputar saat gun berputar
supaya radar fokus scan musuh
    AdjustRadarForGunTurn = true;
    // =====
    // LOOP UTAMA BOT INCESS
    // =====
    while (IsRunning)
    {
        // Radar terus berputar 360 derajat
        SetTurnRadarRight(360);
        // =====
        // GERAKAN ZIGZAG
        // =====
        // ZIG KANAN
        // Bot maju 180 pixel
        SetForward(180);
        // Sambil belok kanan 40 derajat
        SetTurnRight(40);
        // Menjalankan perintah movement
        Go();
        // ZIG KIRI
        // Bot maju lagi
        SetForward(180);
    }
}

```

```

        // Belok kiri 80 derajat
        SetTurnLeft(80);
        // Jalankan movement
        Go();
        //ZIG KANAN
        // Bot maju
        SetForward(180);
        // Belok kanan lebih tajam
        SetTurnRight(80);
        // Jalankan movement
        Go();
        // ZIG KIRI
        // Bot maju
        SetForward(180);
        // Belok kiri lagi
        SetTurnLeft(80);
        // Jalankan movement
        Go();
    }
}
// =====
// EVENT SAAT MUSUH TERLIHAT
// =====
// Method ini akan terus dijalankan saat radar
menemukan musuh
public override void OnScannedBot(ScannedBotEvent e)
{
    // =====
    // HEURISTIK GREEDY
    // =====
    // Jika musuh terlihat maka:
    // 1. Fokus penuh ke musuh
    // 2. Lock radar
    // 3. Lock gun
    // 4. Menghadap musuh
    // 5. Menembak damage maksimum
    // 6. Mendekati musuh
    // Menghitung jarak bot ke musuh
    double distance = DistanceTo(e.X, e.Y);
    // =====
    // RADAR LOCK
    // =====
    // Menghitung sudut radar ke musuh
    double radarTurn = DirectionTo(e.X, e.Y) -
RadarDirection;

```



```

        // Radar diputar ke arah musuh

SetTurnRadarLeft(NormalizeRelativeAngle(radarTurn));
    // =====
    // GUN LOCK
    // =====
    // Menghitung sudut gun ke musuh
        double gunTurn = DirectionTo(e.X, e.Y) -
GunDirection;
    // Gun diputar ke arah musuh
SetTurnGunLeft(NormalizeRelativeAngle(gunTurn));
    // =====
    // BODY LOCK
    // =====
        // Body ikut menghadap musuh supaya agresif
melawan target
    // Menghitung arah body ke musuh
        double bodyTurn = DirectionTo(e.X, e.Y) -
Direction;
    // Body diputar ke arah musuh
SetTurnLeft(NormalizeRelativeAngle(bodyTurn));
    // =====
    // HIGH DAMAGE FIRE
    // =====
    // Jika gun hampir tepat ke musuh, langsung tembak
dengan power maksimum
    // Jika sisa sudut gun kecil
    if (Math.Abs(GunTurnRemaining) < 8)
    {
        Fire(3); //Damage maksimum
    }
    // =====
    // MENDEKATI MUSUH
    // =====
    // Jika musuh jauh bot akan mendekati musuh
    if (DistanceTo(e.X, e.Y) > 150)
    {
        SetForward(120); //Maju ke arah musuh
    }
}
// =====
// EVENT KENA DINDING
// =====
public override void OnHitWall(HitWallEvent e)
{

```

```

        // Membalik arah gerakan
        ReverseDirection();
        // Belok supaya keluar dari dinding
        SetTurnRight(90);
        // Maju menjauh dari dinding
        SetForward(150);
        // Menjalankan movement
        Go();
    }
    // =====
    // EVENT TABRAK MUSUH
    // =====
    public override void OnHitBot(HitBotEvent e)
    {
        // Saat dekat musuh langsung tembak maksimum
        // Fire power maksimum
        Fire(3);
        // Tetap maju agar terus menekan lawan
        SetForward(100);
    }
    // =====
    // EVENT KENA PELURU
    // =====
    public override void OnHitByBullet(HitByBulletEvent e)
    {
        // =====
        // DODGE ZIGZAG
        // =====
        // Saat terkena peluru, bot mengubah arah secara
cepat agar sulit ditembak
        // Belok kanan
        SetTurnRight(50);
        // Maju
        SetForward(100);
        // Jalankan movement
        Go();
        // Belok kiri tajam
        SetTurnLeft(100);
        // Maju lagi
        SetForward(100);
        // Jalankan movement
        Go();
    }
    // =====
    // METHOD REVERSE DIRECTION

```

```

// =====
// Method untuk membalik arah gerakan bot
private void ReverseDirection()
{
    // Jika sebelumnya bot maju
    if (movingForward)
    {
        // Bot mundur
        SetBack(150);
        // Status diubah menjadi mundur
        movingForward = false;
    }
    else
    {
        // Jika sebelumnya bot mundur
        // maka bot maju
        SetForward(150);
        // Status diubah menjadi maju
        movingForward = true;
    }
}
}

```

4.1.3 Penjelasan Komponen Kunci Implementasi Incess

a. AdjustGunForBodyTurn dan AdjustRadarForGunTurn

Digunakan sebagai dasar dalam pembuatan desain bot supaya body, gun, dan radar bergerak secara mandiri. Sehingga bot bisa melakukan beberapa aksi sekaligus dalam satu turn: body bergerak mengejar musuh, gun tetap mengarah ke target untuk menjaga akurasi tembakan, dan radar terus melakukan pemindaian arena. Jika AdjustGunForBodyTurn dan AdjustRadarForGunTurn ini tidak digunakan maka pergerakan body bot akan ikut menggeser arah gun dan radar sehingga penguncian target tidak stabil dan akurasi menurun.

b. Radar Scan 360° pada Loop Utama

Perintah SetTurnRadarRight(360) yang ditempatkan pada loop utama berfungsi untuk mencari target secara terus-menerus. Selama belum ada musuh yang terdeteksi, radar akan berputar penuh untuk melakukan scan ke seluruh

arena untuk mencari musuh. Ketika event *OnScannedBot* aktif, radar kemudian dialihkan untuk mengikuti posisi musuh agar target tetap terpantau.

c. Keputusan Greedy di *OnScannedBot*

Pada *OnScannedBot* digunakan untuk bot dalam mengambil keputusan. Pertama kita harus menjaga musuh untuk tetap berada dalam jangkauan radar melalui radar lock. Gun diarahkan ke target agar siap menembak, lalu body diputar menghadap musuh untuk mendukung mobilitas dan posisi tempur. Jika arah gun sudah akurat, bot akan memanfaatkan peluang dengan menembak menggunakan *Fire(3)*. Terakhir, bot mengatur jarak dengan bergerak mendekat ketika musuh berada terlalu jauh lalu kalau sudah dekat tabrak musuh sambil menembak musuh.

d. Threshold 8 Derajat untuk Menembak

Batas *GunTurnRemaining* < 8 digunakan supaya bot tidak harus menunggu arah gun benar-benar tepat sebelum menembak. Jika derajat *GunTurnRemaining* terlalu kecil, bot akan lama dalam menunggu dan kesempatan untuk menyerang menghilang. Tetapi apabila derajatnya terlalu besar membuat tembakan lebih sering meleset sehingga tidak terkena musuh dan hanya membuang energi bot itu sendiri. Dengan menggunakan delapan derajat, bot tetap dapat menembak secara agresif sambil mempertahankan tingkat akurasi yang cukup baik.

e. Mekanisme *ReverseDirection*

Variabel *movingForward* digunakan untuk menyimpan arah gerak bot saat ini. Ketika bot menabrak dinding, fungsi *ReverseDirection()* akan membalik nilai state tersebut sehingga arah gerak berubah ke sisi berlawanan. Sehingga solusi sederhana untuk menangani tabrakan dengan arena. Walaupun tidak melibatkan logika navigasi yang rumit, pendekatan ini sudah efektif untuk mencegah bot terjebak di dinding atau sudut arena dan menjaga pergerakannya tetap berlangsung.

4.2 Struktur Data yang digunakan dalam Program Bot *Incess*

Pemilihan struktur data pada bot **Incess** bertujuan untuk mendukung pergerakan agresif, lock target, dan pengambilan keputusan cepat berdasarkan strategi *greedy aggressive pursuit*.

1. *movingForward* : bool

- **Tipe:** Boolean (true / false).

- **Deskripsi:**

Digunakan untuk menyimpan status arah gerakan bot supaya bisa mengetahui posisi musuh sedang maju atau mundur. Variabel ini digunakan pada method `ReverseDirection()` agar bot dapat membalik arah ketika menabrak dinding atau perlu menghindari serangan lawan.

2. **distance : double**

- **Tipe:** Double (bilangan desimal).

- **Deskripsi:**

Digunakan untuk menyimpan jarak antara bot dan musuh yang terdeteksi radar. Nilai ini digunakan untuk menentukan apakah bot harus mendekati musuh atau tetap menyerang dari jarak tertentu.

```
double distance = DistanceTo(e.X, e.Y);
```

3. **RadarTurn : double**

- **Tipe:** Double.

- **Deskripsi:**

Menyimpan besar sudut rotasi radar menuju arah musuh. Digunakan untuk membuat radar tetap terkunci (*radar lock*) pada target agar musuh terus terdeteksi.

```
double radarTurn = DirectionTo(e.X, e.Y) - RadarDirection;
```

4. **GunTurn : double**

- **Tipe:** Double.

- **Deskripsi:**

Menyimpan sudut rotasi gun menuju posisi musuh. Digunakan agar arah tembakan tetap fokus dan akurat ke target.

```
double gunTurn = DirectionTo(e.X, e.Y) - GunDirection;
```

5. **BodyTurn : double**

- **Tipe:** Double.

- **Deskripsi:**

Digunakan untuk menghitung arah putaran body bot menuju musuh. Struktur data ini membantu bot bergerak agresif mengejar lawan.

```
double bodyTurn = DirectionTo(e.X, e.Y) - Direction;
```

6. ScannedBotEvent e

- **Tipe:** Objek Event.
- **Deskripsi:**

Objek event yang menyimpan informasi musuh saat radar mendeteksi lawan, seperti:

 - posisi musuh (X, Y),
 - Energi,
 - Arah,
 - dan jarak musuh.
- Data dari event ini digunakan sebagai dasar pengambilan keputusan greedy pada bot.

7. HitWallEvent e

- **Tipe:** Objek Event.
- **Deskripsi:**

Digunakan ketika bot menabrak dinding arena. Event ini memicu bot untuk membalik arah dan keluar dari area dinding agar tidak terjebak.

8. HitBotEvent e

- **Tipe:** Objek Event.
- **Deskripsi:**

Menyimpan informasi ketika bot bertabrakan langsung dengan musuh. Event ini digunakan untuk menjalankan strategi agresif berupa menembak dengan power maksimum dan terus menekan lawan dari jarak dekat.

9. HitByBulletEvent e

- **Tipe:** Objek Event.
- **Deskripsi:**

Digunakan ketika bot terkena peluru musuh. Event ini memicu gerakan dodge zigzag agar bot lebih sulit terkena serangan berikutnya.

10. Struktur Event-Driven

- **Tipe:** Struktur berbasis event (*event-driven programming*).
- **Deskripsi:**

Program bot berjalan berdasarkan event yang terjadi selama pertandingan, seperti:

 - musuh terdeteksi,
 - terkena peluru,
 - menabrak dinding,
 - atau bertabrakan dengan musuh.

Pendekatan ini membuat bot dapat merespons kondisi arena secara cepat dan realtime tanpa perlu menyimpan banyak data historis.

4.3 Analisis Desain Solusi Algoritma *Greedy* Melalui Skenario Pengujian

Bagian ini digunakan untuk melakukan analisis terhadap desain solusi algoritma greedy yang diterapkan pada bot *incess* dengan memprediksi perilaku bot dalam beberapa kondisi pertandingan yang berbeda. Analisis ini dilakukan untuk mengetahui seberapa efektif strategi greedy yang digunakan dalam menghadapi berbagai situasi selama permainan berlangsung.

Strategi greedy pada bot *incess* dirancang untuk selalu memilih aksi yang dianggap paling menguntungkan pada saat itu juga. Bot akan langsung mengejar musuh yang terdeteksi, mengunci radar dan gun ke arah target, menembak menggunakan damage maksimum, serta bergerak zigzag untuk mengurangi kemungkinan terkena peluru.

Melalui beberapa skenario pengujian berikut, dapat dianalisis bagaimana strategi greedy tersebut bekerja, kapan strategi berjalan efektif, dan kondisi apa saja yang dapat menyebabkan performa bot menjadi kurang optimal.

4.3.1 Skenario Pengujian Bot Asisten

- **Skenario 1: Bot *Incess* melawan Bot *Crazy* (sampel)**

Dari awal pertandingan mulai Bot *Incess* langsung fokus mengejar bot *Crazy* sambil bergerak zigzag dan nembak pakai power yang maksimal. Sementara itu, *Crazy* tidak mempunyai pola serangan yang jelas. Dia cuma nembak kecil kalau radar kebetulan nemu *IncessBot*, jadi damage yang masuk ke *IncessBot* hampir tidak terasa.

Gerakan bot *Crazy* sebenarnya membuat bot *Incess* sedikit susah karena dia sering belok-belok besar dan kadang tiba-tiba berubah arah setelah nabrak dinding. Tapi karena *Crazy* tidak mempunyai gerakan menghindar yang bagus dan hanya muter balik secara pasif, bot *Incess* tetap mudah buat ngunci dan ngejar posisinya lagi.

Pada akhirnya selama 3 kali pertandingan yang menang bot *Incess* karena damage tembakannya jauh lebih sakit dibanding tembakan kecil punya *Crazy*. Gerakan zigzag *IncessBot* juga bikin serangan *Crazy* sering meleset.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	incess 1.0	2022	500	100	816	121	367	117	10	0	0
2	Crazy 1.0	320	0	0	152	0	168	0	0	10	0

- **Skenario 2 : Bot *Incess* melawan Bot *Velocity* (sampel)**

Bot *Velocity* mempunyai pola gerak yang sebenarnya berulang yaitu maju pelan beberapa saat lalu mundur lebih cepat. Tapi karena dia sering ganti arah dan langsung belok saat kena tembak, gerakannya jadi lumayan susah ditebak.

Akibatnya, IncessBot harus kerja lebih keras buat ngejar dan tidak bisa langsung mengunci posisi lawan.

Selama dikejar, bot Velocity terus nembak kecil setiap kali berhasil scan musuh. Damage-nya memang tidak besar, tapi karena tembakannya rutin sehingga energi bot Incess lama-lama tetap berkurang sedikit demi sedikit. Ditambah lagi, IncessBot yang terlalu fokus ngejar jadi nggak bisa banyak menghindar.

Walaupun begitu, bot Velocity sebenarnya tidak terlalu kuat buat bertahan lama. Serangannya lemah dan dia lebih banyak kabur daripada menyerang. Begitu Ibot ncess berhasil menabrak dia sampai ke dinding atau menangkap momen saat bot Velocity baru ganti arah, bot Incess bisa langsung menyerang dari jarak dekat dengan tembakan power besar. Pada akhirnya selama 3 kali pertandingan yang menang bot Incess cuma pertandingannya lebih lama dan energi IncessBot lebih banyak terkuras dibanding waktu lawan Crazy karena proses kejar-kejarannya cukup melelahkan.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	incess 1.0	2291	500	100	864	187	575	64	10	0	0
2	Velocity Bot 1.0	692	0	0	240	0	452	0	0	10	0

- **Skenario 3 : Bot Incess melawan SpinBot (sampel)**

SpinBot melakukan pergerakan dengan cara terus memutar secara terus menerus sambil bergerak melingkar, dan setiap kali radarnya ketemu bot Incess dia langsung nembak pakai power paling besar, yaitu Fire(3). Karena putarannya terus jalan tanpa henti, bot Incess jadi cukup sering kena tembakan saat mencoba mendekat.

Walaupun bot Incess bergerak zigzag supaya lebih susah ditembak, tetap saja terkadang bot incess terkena serangannya. SpinBot langsung mengeluarkan Fire(3) lagi dari jarak dekat, jadi damage yang diterima IncessBot makin besar tetapi kalau sudah dikunci kemudian di tabrak oleh bot Incess dia akan sulit untuk mengeluarkan fire nya sehingga damage yang terkena bot incess hanya sedikit. Setelah beberapa saat, IncessBot akhirnya bisa membaca arah gerakannya dan mencari posisi serangan yang lebih enak buat menyerang balik. Pada akhirnya selama 3 kali pertandingan yang menang bot Incess tapi ini jadi pertarungan paling berat.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	incess 1.0	2251	500	100	896	187	504	63	10	0	0
2	Spin Bot 1.0	392	0	0	208	0	184	0	0	10	0

- **Skenario 4 : Bot Incess melawan Bot Trackfire (sampel)**

Dalam pertandingan melawan TrackFire, bot incess punya keunggulan yang cukup jelas. TrackFire hanya memutar radar dan gun terus-menerus sampai berhasil menemukan musuh, lalu baru menembak. Pola ini membuat serangannya kurang efektif karena dia tidak langsung menekan lawan sejak awal dan sangat bergantung pada hasil scan. Selama TrackFire masih sibuk mencari target, bpot Incess sudah lebih dulu bergerak agresif untuk mendekat dan menyerang. Akibatnya, TrackFire sering terlambat memberi perlawanan karena baru menembak setelah musuh terdeteksi, sementara Bot Incess sudah lebih dulu menguasai jalannya pertandingan. Pada akhirnya selama 3 kali pertandingan yang menang bot Incess karena mampu menyerang lebih cepat dan memberi tekanan sebelum TrackFire sempat membangun serangan yang efektif.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Incess 1.0	2553	500	100	928	210	704	111	10	0	0
2	Track Fire 1.0	598	0	0	598	0	0	0	0	10	0

- **Skenario 5 : Bot Incess melawan Corners Bot (sampel)**

Saat melawan bot Incess kelemahan paling besar bot ini ada di cara gerakannya yang hanya mengikuti dinding arena. Kalau di awal posisi musuh masih di tengah, dia akan langsung bergerak menuju pinggir atau sudut arena. Masalahnya, selama proses itu dia jadi mudah ditebak dan tidak punya banyak ruang untuk menghindar. Bot incess bisa langsung mengejar dan bahkan menabraknya sebelum bot corners sempat siap menyerang.

Karena terlalu fokus bergerak ke sudut dan hanya berputar di area dinding, bot corners juga kehilangan banyak kesempatan untuk memberi tekanan ke lawan. Serangannya jadi terlambat dan tidak efektif. Pada akhirnya selama 3 kali pertandingan yang menang bot Incess karena bot incess bisa terus mendominasi pertandingan sejak awal tanpa memberi ruang bagi bot corners untuk membangun serangan balik.

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Incess 1.0	2255	500	100	816	156	584	98	10	0	0
2	Corners 1.0	288	0	0	286	0	2	0	0	10	0

- **Skenario 6 : Bot Incess melawan Berbagai Bot**

Setelah melawan banyak jenis bot dengan pola permainan yang berbeda-beda, IncessBot tetap menang. Karena IncessBot menggunakan gaya bermain yang sangat agresif dan terus memberi tekanan kepada lawan sejak awal

pertandingan. Saat radar menemukan musuh, IncessBot langsung mengunci arah radar, gun, dan body ke target sehingga lawan sulit kabur dari pantauan. Dengan fokus serangan yang terus terjaga, IncessBot bisa langsung mengejar dan menyerang tanpa memberi waktu bagi lawan untuk membangun strategi.

Selain itu, IncessBot selalu menembak menggunakan damage maksimum dan memanfaatkan tabrakan untuk mempercepat pengurangan energi lawan. Strategi ini membuat banyak bot kesulitan bertahan. IncessBot juga terus mendekati musuh agar serangannya lebih sering mengenai sasaran dan tekanan tetap terasa sepanjang pertandingan. Kekurangan dari IncessBot ini salah satunya yaitu IncessBot selalu menggunakan damage maksimum saat menembak, sehingga energi miliknya lebih cepat berkurang, apalagi kalau banyak tembakan yang meleset.

a. Hasil Pertandingan Pertama

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Incess 1.0	4632	1950	240	1440	248	662	92	3	0	0
2	Velocity Bot 1.0	2989	2100	240	452	36	160	1	1	0	1
3	Walls 1.0	2725	2000	0	640	28	54	3	1	1	0
4	ENAA_Nemesis 1.0	2712	1650	0	783	75	204	0	0	1	2
5	Track Fire 1.0	2217	1250	0	908	59	0	0	0	1	0
6	Fire 1.0	2072	1750	0	320	0	2	0	0	0	0
7	Template Bot 1.0	1847	1600	0	154	0	74	18	0	0	0
8	Crazy 1.0	1748	1500	0	196	0	52	0	0	0	1
9	Spin Bot 1.0	1743	1150	0	448	3	142	0	0	1	0
10	Corners 1.0	1574	1300	0	270	2	2	0	0	0	1
11	Ram Fire 1.0	1538	800	0	430	13	258	36	0	1	0
12	My First Leader 1.0	1414	1400	0	0	0	14	0	0	0	0
13	AyamJago 1.0	1297	1000	0	174	3	119	0	0	0	0

b. Hasil Pertandingan Kedua

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Incess 1.0	4239	1700	0	1472	232	742	92	2	2	0
2	Spin Bot 1.0	3394	2250	0	992	84	67	0	0	1	2
3	Velocity Bot 1.0	2632	1950	0	396	10	276	0	0	0	2
4	Walls 1.0	2439	1700	120	560	26	32	0	1	0	0
5	AyamJago 1.0	2388	1950	120	254	15	48	0	1	1	0
6	ENAA_Nemesis 1.0	2373	1450	240	542	28	113	0	1	1	0
7	Track Fire 1.0	2244	1300	0	900	43	0	0	0	0	0
8	Crazy 1.0	1807	1550	0	196	0	61	0	0	0	1
9	Ram Fire 1.0	1656	1000	0	358	9	253	35	0	0	0
10	Corners 1.0	1628	1400	0	224	0	4	0	0	0	0
11	Fire 1.0	1556	1200	0	348	0	7	0	0	0	0
12	My First Leader 1.0	1308	1300	0	0	0	8	0	0	0	0
13	Template Bot 1.0	932	700	0	128	0	104	0	0	0	0

c. Hasil Pertandingan Ketiga

Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	Incess 1.0	3334	1250	0	1200	116	659	109	2	1	1
2	Track Fire 1.0	3213	2050	0	1112	51	0	0	0	1	2
3	Spin Bot 1.0	3192	1950	240	784	85	132	1	2	0	0
4	ENAA_Nemesis 1.0	2535	1650	0	636	15	234	0	0	1	0
5	Crazy 1.0	2496	2200	0	248	4	42	2	0	0	1
6	Walls 1.0	2309	1650	120	480	26	32	0	0	2	1
7	Velocity Bot 1.0	2013	1450	0	316	7	240	0	0	0	0
8	My First Leader 1.0	1713	1700	0	0	0	13	0	0	0	0
9	Ram Fire 1.0	1663	1050	0	352	6	202	53	0	0	0
10	Template Bot 1.0	1423	1050	120	174	21	58	0	1	0	0
11	Fire 1.0	1196	850	0	320	17	8	0	0	0	0
12	AyamJago 1.0	1080	850	0	108	0	122	0	0	0	0
13	Corners 1.0	989	750	0	234	0	5	0	0	0	0

4.3.2 Ringkasan Kekuatan dan kelemahan Desain *Greedy Inces*

- **Kekuatan Bot**

Bot ini memiliki kecepatan dalam mengambil keputusan yang sangat baik karena strategi greedy yang digunakan sederhana dan tidak membutuhkan perhitungan sangat rumit. Bot hanya melakukan pengecekan kondisi secara langsung tanpa proses yang berat, sehingga dapat bergerak dan merespons dengan cepat tanpa risiko terkena *turn timeout*.

Selain itu, bot memiliki gaya permainan yang sangat agresif. Ketika musuh terdeteksi, bot akan melakukan tindakan dengan cara langsung mengunci radar dan gun ke arah target lalu menembak sambil menabrak musuh menggunakan power maksimum. Pola ini membuat tekanan terhadap lawan terus berlangsung dan damage yang diberikan juga cukup besar selama pertandingan.

Pergerakan zigzag yang dijalankan terus-menerus pada loop utama juga menjadi kelebihan tersendiri. Gerakan ini membuat arah pergerakan bot lebih sulit ditebak oleh musuh, terutama bot lawan yang menggunakan prediksi gerakan lurus (*linear prediction*).

- **Kelemahan Bot**

Walaupun bot ini memiliki banyak kekuatan tetapi bot ini juga masih memiliki beberapa kekurangan. Salah satunya adalah belum adanya sistem prediksi posisi musuh. Bot hanya menembak berdasarkan posisi musuh saat itu juga, sehingga tembakan sering meleset jika lawan bergerak cepat atau berubah arah secara tiba-tiba.

Bot juga belum memiliki kemampuan dalam mengatur energi yang baik. Semua tembakan selalu menggunakan Fire(3) atau power maksimum tanpa mempertimbangkan sisa energi yang dimiliki. Akibatnya, energi bot bisa cepat habis dan membuat bot menjadi lebih lemah di akhir pertandingan.

Dalam pertarungan dengan banyak musuh, bot belum mampu menentukan prioritas target. Bot hanya fokus pada musuh yang terdeteksi radar dan mengabaikan ancaman lain di arena. Hal ini bisa berbahaya karena serangan dari arah lain sering tidak ditangani.

Selain itu, dodge saat terkena peluru masih sederhana dan kurang adaptif. Bot selalu bergerak dengan pola belok yang hampir sama tanpa memperhatikan arah datangnya peluru. Karena polanya mudah dikenali, lawan dapat lebih mudah memprediksi gerakan bot setelah beberapa waktu.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Algoritma greedy pada bot Incess dalam permainan Robocode Tank Royale berhasil diterapkan melalui strategi **Aggressive** dan **Bullet Damage Maksimal**. Bot mengambil keputusan secara cepat berdasarkan kondisi yang terjadi pada setiap turn, seperti mengunci radar dan gun ke arah musuh, bergerak mendekati lawan, menembak menggunakan daya maksimum, menabrak musuh, serta melakukan pergerakan zigzag untuk menghindari serangan.

Selain itu, strategi greedy yang kita gunakan membuat bot mampu memberikan tekanan yang besar kepada lawan dan menjaga target tetap berada dalam jangkauan radar. Pergerakan zigzag serta mekanisme respons terhadap tabrakan dan serangan membantu bot untuk bertahan lebih lama selama pertandingan itu berlangsung.

Namun, pendekatan greedy pada bot Incess juga memiliki keterbatasan karena seluruh keputusan hanya berfokus pada keuntungan sesaat tanpa mempertimbangkan strategi jangka panjang. Bot belum memiliki kemampuan prediksi posisi musuh, pengelolaan energi yang optimal, maupun sistem prioritas target saat menghadapi banyak lawan. Sehingga digunakan **Fire(3)** secara terus-menerus dan pola gerak yang relatif tetap dapat menyebabkan energi cepat habis serta membuat performa kurang optimal terhadap musuh yang bergerak cepat atau menyerang dari berbagai arah.

5.2 Saran

Bot Incess dapat ditingkatkan dengan menambahkan untuk dilakukan prediksi gerakan musuh agar akurasi tembakan kepada lawan bisa meningkat, khususnya ketika menghadapi lawan yang bergerak cepat atau sering berubah arah. Dan juga kekuatan tembakan sebaiknya dibuat lebih adaptif dengan mempertimbangkan jarak musuh dan sisa energi bot sehingga konsumsi energi menjadi lebih efisien.

Bot juga disarankan memiliki mekanisme pemilihan prioritas target dalam pertandingan multibot, sehingga tidak hanya fokus pada satu musuh yang sedang dipindai radar tetapi juga mampu mempertimbangkan ancaman lain di arena. Di sisi pergerakan, pola dodge dapat dikembangkan menjadi lebih dinamis dan menyesuaikan arah datangnya peluru agar tidak mudah diprediksi lawan.

Apabila semua kekurangan dari bot Incess ini dapat diperbaiki maka tidak hanya mempertahankan keunggulan kecepatan pengambilan keputusan dari algoritma greedy, tetapi juga memiliki strategi yang lebih adaptif, efisien, dan kompetitif dalam berbagai kondisi pertandingan Robocode Tank Royale.

LAMPIRAN

A. Repository Github:

[https://github.com/isnaini75-star/TUBES_Tabolabale.](https://github.com/isnaini75-star/TUBES_Tabolabale)

B. Video Penjelasan:

<https://youtu.be/sqST0mIpess>

DAFTAR PUSTAKA

Roihan, A., Nasution, K., & Siambaton, M. Z. (2022). Implementasi algoritma greedy kombinasi dengan perulangan pada aplikasi penjadwalan praktikum. *Sudo Jurnal Teknik Informatika*, 1(2), 42-50.

Darnita, Y., & Toyib, R. (2019). Penerapan Algoritma Greedy Dalam Pencarian Jalur Terpendek Pada Instansi-Instansi Penting Di Kota Argamakmur Kabupaten Bengkulu Utara. *Jurnal Media Infotama*, 15(2).

Purnama, R. D. S., Nisa, F., Tundo, T., Nurohman, K., Fakhurrofi, F., Nugrahaini, L., & Dalail, D. (2024). Implementasi penggunaan algoritma greedy best first search untuk menentukan rute terpendek dari cilacap ke yogyakarta. *Jurnal Informatika dan Teknik Elektro Terapan*, 12(2).